PyLCF — User Guide

Graphical interface for Linear Combination Fitting (LCF) of spectra and other 1D data Version 1.0.0 · Program: `pylcf` (run: `python -m pylcf`)

> **Note.** The program's buttons and labels are in **English**. This guide refers to each control by its on-screen label in quotation marks "...".

---

1. What does it do?

LCF stands for *Linear Combination Fitting*. **PyLCF** is the accompanying graphical application.

A measured dataset — a curve $y(x)$ sampled over some axis x (energy, 2θ, time, wavelength, m/z …) — is described as a **weighted sum** of reference datasets on the same axis:

$$y_{\mathrm{meas}}(x) \;\approx\; \Sigma_{\mathrm{i}}\, \mathrm{w}_{\mathrm{i}} \cdot y_{\mathrm{i}}(x)$$

The program finds the weights w_{i}. For area-normalized data and in *convex* mode these weights are read directly as **fractions** (all $\mathrm{w}_{\mathrm{i}} \geq 0$, sum = 1) — e.g. population fractions.

The running example throughout this guide is an **NIS/NRVS spectrum** decomposed into its component PVDOS (the Fe-PVDOS of different species or protonation states from DFT). The method is **not limited to spectra**, though: it fits any data that is plausibly a **linear combination** of references on a common x-axis — e.g. XAS/XANES LCF, diffraction patterns, chromatograms or kinetic traces.

> **What it is not:** not a general (nonlinear) curve fit and not peak fitting. The references are only scaled in **amplitude** (not shifted or stretched along x); they should cover comparable x-ranges (different grids are allowed and are interpolated).

The GUI and the command-line interface (`pylcf-cli`) share the same numeric core (`pylcf/core.py`); the results are therefore identical by construction. The graphical interface only changes the *operation* — above all the data entry. A control run on the test data reproduces the same R-factor ($1.614 \cdot 10^{-3}$) as the original script.

---

2. Installation & launch

Requirements

Package	Purpose	Required?
Python ≥ 3.9	runtime	yes
`numpy`, `scipy`, `matplotlib`	computation & plot	yes
`tkinter`	graphical interface	yes (bundled with the standard Python installer on Windows/macOS)
`pandas`, `openpyxl`	only for the Excel export (.xlsx)	optional

Install the packages (in PowerShell):

```
pip install numpy scipy matplotlib pandas openpyxl
```

> On Linux, Tkinter is sometimes missing and is installed separately (`sudo apt install python3-tk`). On Windows this is not necessary.

Start the program

Straight from the repository (no installation needed):

```
python -m pylcf
```

After `pip install .` the GUI is available as the `pylcf` command and a command-line interface as `pylcf-cli` (or `python -m pylcf.cli`). If Tkinter or Matplotlib is missing, the program prints a notice on startup and exits.

> **Scrolling the control column.** The left column (sections 1–4) can be taller than the window — for instance on small screens or with many references. It therefore has a **scrollbar** on the right; use it (or the **mouse wheel** over the column) to reach every button, including "Interactive overlay" and "Show guide" at the bottom of section 4. The window is also freely resizable.

---

3. Entering data

There are four ways. **Variant A (pasting from Excel)** and **Variant B (importing an Excel file)** are the most convenient. If all spectra lie on the **same** energy grid, use pasting (A) or the Excel import in shared-grid mode (B). If the spectra have **different x/y values** (their own grid each), use the Excel import in XY-pairs mode (B), the **folder import** (C, one file per spectrum), or load individual files (D) — in all of these the fit automatically interpolates onto the common overlap region.

Variant A — paste a table from Excel

1. In Excel, select the columns — preferably with a header row, e.g.

Energie	Gemessen	WT	DM	Red
0	0.000	0.000	0.000	0.000
2	0.012	0.015	0.010	0.008
…	…	…	…	…

2. Press **Ctrl + C**.
3. In the program click "Paste table …" → "Paste from clipboard".
4. Click "Detect columns". The program shows a preview and assigns each column a **role** (drop-down):
   - **Energy** — the energy axis (cm⁻¹). Exactly **one** column.
   - **Measured** — the spectrum to be described. **At most one**.
   - **Reference** — a component / sub-spectrum. **Any number**.
   - **Ignore** — column is not used.

   The column **names** can still be edited here.
5. Click "Apply". The spectra appear in the list under "1 - Data".

Decimal commas and delimiters are detected automatically: when copied from Excel the columns are tab-separated, and a comma is interpreted as a decimal separator (German Excel). If detection fails, you can force it in the dialog via the fields "Delimiter" (tab / semicolon / comma / space) and "Decimal" (point / comma).

> **Note:** when pasting, all columns share the **same** energy axis. For spectra on *different* energy grids, use Variant B ("XY pairs") or Variant C.

Variant B — import an Excel file (.xlsx/.xlsm)

Instead of copying, an Excel file can be read directly — including workbooks with several sheets.

1. Click "Excel file …" and pick the `.xlsx` / `.xlsm` file.
2. In the dialog choose the **sheet** and the **layout**:
   - **Shared grid** — one **Energy** column + any number of intensity columns (**Measured** / **Reference**), exactly like Variant A. All spectra share one grid.
   - **XY pairs** — consecutive column pairs `E, I, E, I, …`. Each pair is a **separate spectrum with its own energy grid**, so the spectra may have **different x/y values** and even different numbers of points (just pad shorter ones with empty cells). A single leftover column without a partner is ignored.
3. Give each column or pair a **role** (Measured / Reference / Ignore). If a column name contains "gemessen"/"measured", that pair is pre-selected as **Measured**. Names are editable.
4. Click "Apply" — the spectra appear in the list (source "Excel").

Numbers are read straight from the cells; German decimal numbers stored as text are also recognized. In XY-pairs mode, rows with empty/invalid cells are dropped **per pair**, so spectra of different length can live on one sheet.

> **Requirement:** the Excel import needs the `openpyxl` package (`pip install openpyxl`). If it is missing, the program says so.

Variant C — import a folder of Excel files

When each spectrum lives in its **own** Excel file, a whole folder can be read at once.

1. Click "Excel folder ..." and pick the folder. Every `.xlsx` / `.xlsm` inside is read (temporary Excel lock files such as `~$…` are skipped).
2. **Each file** becomes one spectrum (first two columns = energy, intensity; extra columns are ignored). The files may have **different energy grids**.
3. Give each file a **role** in the dialog (Measured / Reference / Ignore). If a filename contains "gemessen"/"measured", that file is pre-selected as **Measured**. Names are editable.
4. Click "Apply" — the spectra land in the list (source "Excel").

Variant D — load files

For classic two-column text files (energy + intensity):

- "Load measured ..." — one file (the measured spectrum).
- "Load references ..." — one or several files (the components).

Supported formats are `.dat`, `.csv`, `.txt`. Comment lines (starting with `# ! % & * / ' "`) are skipped; allowed delimiters are space, comma, tab and semicolon. For **German decimal commas** tick the box "Decimal comma (files)" beforehand. As with the XY pairs, the files may have different grids — they are automatically interpolated onto the common overlap window.

Changing roles afterwards

Select a row in the list and press "Switch role" to toggle between *measured* and *reference*, or "Remove" to delete it. **Double-click** a row (or **"Rename"**) to give it a custom name; spectra without a name are numbered automatically ("Reference 1", "Reference 2", ...). The last folder and your settings are remembered between sessions. Only **one** measured spectrum is active at a time; if a second one is added as "measured", the previous one automatically becomes a reference.

---

4. Options (section "2 - Options")

Mode ("Mode")

Mode	Constraint on the weights	typical use
convex	$w_i \geq 0$ **and** $\Sigma_i w_i = 1$	population fractions (default)
nnls	$w_i \geq 0$	non-negative amplitudes, no sum constraint
linear	none	unconstrained least squares; may give **negative** (unphysical) weights
all	—	computes convex/nnls/linear side by side; the plot shows *convex*

> **On interpretation:** a smaller R-factor does **not** automatically mean the better model. *linear* has the most degrees of freedom and almost always reaches the smallest R-factor — even when the weights are physically meaningless. For population fractions, *convex* is the right choice.

Normalization ("Normalization")

- **area** — area = 1. Only this makes the *convex* weights true fractions. **Recommended.**
- **max** — maximum = 1.
- **none** — no normalization (spectra are used as they are).

> **Note:** *area* requires a **positive** net area. If a baseline-subtracted curve has as much negative as positive area (net area ≤ 0), the program reports an error — then use *max*/*none* or restrict the window to a positive region.

Energy window

"Energy min / max" limit the fit range (in cm^{-1}). Leave both empty = automatic overlap of all spectra. Useful, for example, to fit only the low-frequency region (soft modes).

Δx weighting (uneven grids)

Default: **off** (every point counts equally — identical to the command line). **On** weights each point by its x-spacing (trapezoidal rule); the fit then approximates the **integral** of $(S_{meas} - S_{fit})^2$ over x and no longer depends on sampling density. Useful when the grid is **uneven** (merged measurements, references sampled at very different densities); for an even grid the effect is negligible. The R-factor, RMSE and R^2 are then computed as weighted (integral) quantities.

Bootstrap

"Bootstrap N" produces error bars on the weights (standard deviation and 95 % confidence interval) by residual bootstrap — e.g. `500`. `0` turns it off. "Seed" makes the result reproducible.

The plain bootstrap resamples *individual* residuals and assumes they are **independent**. Spectral residuals are neighbour-**correlated**, so these intervals tend to **underestimate** the uncertainty. The **"Block bootstrap"** option instead resamples contiguous residual blocks (~√n), preserving short-range correlation and giving more honest (usually wider) intervals. For *convex*, weights near 0 or 1 are pinned at the boundary — there the distribution is one-sided and the ± value (std) is less meaningful than the percentile interval.

Convenience

Tooltips: hover over "Mode", "Normalization", "Energy min" or "Bootstrap" for a short explanation. **Shortcuts:** `F5` = fit, `Ctrl+S` = export fit data, `Ctrl+O` = Excel file, `Ctrl+P` = paste table, `Esc` closes dialogs. The last used directory is remembered.

---

5. Reading the result (section "3 - Result")

After "Run fit" you get:

- a **table** with weight/fraction per reference (± and 95 % interval if bootstrap was active, plus the F-test p-value), plus the sum of the weights;
- the **goodness-of-fit measures**:
- **R-factor** = $\frac{\Sigma\,(S_{meas} - S_{fit})^2}{\Sigma\,S_{meas}^{\,2}}$ — smaller is better (relative residual);
- **RMSE** — root mean square error (absolute);
- **R²** — coefficient of determination (1 = perfect).

If bootstrap is active, components whose 95 % interval **includes 0** are greyed out and marked **"~0"** — not statistically supported and possibly unnecessary.

The **"p (F)"** column shows a **drop-one F-test**: each reference is refitted *without* that component to test whether its removal significantly worsens the fit. `p < 0.05` => the component is justified; larger values (grey row) => it adds nothing reliable. **Important:** the F-test assumes **independent** residuals. Spectral residuals are strongly correlated — the result reports the lag-1 autocorrelation ρ and the *effective* sample size N_eff. The p-values are therefore **too optimistic** and should be read only as a relative guide; the test is rigorous for *linear* and approximate for *nnls*/*convex*. The interface now **also shows the p-value corrected with N_eff**; both p-values, ρ and N_eff are written to the exports (.dat/JSON/Excel). The more trustworthy uncertainty is the (block-)bootstrap interval.

The **plot** shows the measured spectrum, the fit and the weighted components on top, and the residual below. The **toolbar** lets you zoom, pan and save the plot directly. The **"X axis"** and **"Y axis"** fields (section "2 - Options") set the **axis labels** freely — e.g. "2θ (°)" and "counts" for other kinds of data; the defaults are "Energy (cm⁻¹)" and "normalized intensity". The **"x column name"** field sets the name of the **x column in the exports** (.dat/Excel) — default "energy".

---

6. Interactive overlay ("Interactive overlay")

The interactive overlay opens a separate window in which each reference can be **scaled up or down** with a **slider**. It shows the measured spectrum, the **sum** of the weighted references ($\Sigma\, w_i \cdot S_i$) and the individual components, with the residual below. This lets you explore **by eye** which mixture fits best.

> **Window layout.** The plot and the controls are separated by a **draggable divider**. Drag it **down** to give the slider area more room (e.g. with many references or on a short screen), or **up** for a larger plot. The window is freely resizable, and the whole slider area is visible when it opens.

Important: the overlay is for **exploration**. The numbers you report should come from the automatic fit (with bootstrap confidence intervals).

Live read-outs (update on every slider move):

- the **fraction** of each component in the total model (w_i / Σw_j, in %);
- the **R-factor** and **R^2** of the current sum.

Slider scaling. Each slider runs from 0 to a **data-driven maximum** — twice the largest of: the auto-fit weight, the "area-match" weight (the component alone carrying the full measured area) and the "peak-match" weight. The area-match floor keeps a slider usable even for a component the fit set to ~0. The sliders are therefore **sensibly scaled** regardless of normalization: with the recommended **area** normalization the values lie in a readable 0 … ≈ 2 range; with **none** the maxima adapt to the raw intensities (correspondingly large numbers).

Next to each slider a **number field** shows the exact value and accepts direct input (decimal comma is fine). A value beyond the current maximum **extends the slider range automatically** — so arbitrary "over-shooting" is possible without making the slider unusable in the normal range.

Buttons:

- **"Load from auto-fit"** — sets the sliders to the *convex* weights of the last fit.
- **"Reset"** — sets all sliders to 0.

Export from the overlay (same formats as the main export):

- **"Save image"** — the figure (.png / .pdf / .svg);
- **"Data (.dat/.csv)"** — energy, measured, fit, residual, weighted components;
- **"Excel (.xlsx)"** — sheets "fit" and "summary";
- **"JSON"** — weights, sums and goodness-of-fit.

> **Note:** the overlay uses the same prepared data as the fit (so the energy window and normalization still apply). After changing the window or normalization, run the fit again and reopen the overlay.

---

7. Export (section "4 - Export")

Button	Format	Contents
"Fit data (.dat/.csv)"	tab-separated (.dat)	energy, measured, fit, residual, weighted components — directly importable into **Origin**
"Fit data (.dat/.csv)"	comma-separated (.csv)	as above
"Excel (.xlsx)"	workbook	sheet "fit" (the columns) + sheet "summary" (weights & goodness-of-fit) — needs `pandas` + `openpyxl`
"Summary (.json)"	JSON	all weights, sums, goodness-of-fit (machine-readable)
"Plot (.png)"	.png / .pdf / .svg	the figure

The `.dat`/`.csv` files carry a **comment header** (lines starting with `#`) with program/version, mode, normalization, fit window and the weights, for reproducibility. Origin and similar tools skip comment lines automatically.

In addition, **"Copy results"** (section "3 - Result") copies the result table with the goodness-of-fit as tab-separated text to the clipboard.

---

8. Common problems

- **"Empty fit window"** — the energy ranges of the spectra do not overlap, or Emin/Emax are set too narrow. Check the limits or clear them.
- **Wrongly parsed numbers** (e.g. off by a factor of 1000, or "NaN") — set the delimiter / decimal separator explicitly in the paste dialog. A common cause: the German comma was mistaken for a column separator, or vice versa.
- **"exactly one measured spectrum"** — check the roles in the list; correct with "Switch role" if needed.
- **Negative weights** — the mode is *linear*. For physical fractions switch to *convex* (or *nnls*).
- **.xlsx export does nothing** — `pandas`/`openpyxl` are not installed (see section 2). The other export formats still work.
- **"area normalization not meaningful"** — the net area is <= 0 (as much negative as positive). Use *max*/*none* or restrict the window to a positive region.
- **Excel cells empty after import** — formula cells need cached values; open and save the file in Excel once.

---

9. Quick test with the sample data

The package includes `beispiel_tabelle.txt` — a ready-made table (tab-separated, German decimal commas, header `Energie Gemessen WT DM Red`). Use it to try Variant A right away:

1. Open the file in a text editor, **select all (Ctrl + A)**, **copy**.
2. In the program: "Paste table …" → paste → "Detect columns".
3. Check the roles (Energy / Measured / 3× Reference) → "Apply".
4. Mode *convex*, normalization *area*, "Run fit".

Expected result: fractions ≈ **WT 0.55 · DM 0.30 · Red 0.15** at an R-factor around $1.4 \cdot 10^{-3}$ — the "true" values built into the data.

---

10. Note: the command-line interface

For **batch processing and scripting**, use the command-line interface `pylcf-cli` (or `python -m pylcf.cli`). The GUI and the CLI share the same core (`pylcf/core.py`) and therefore give the same weights and goodness-of-fit. The CLI writes `.dat` and `.json`, optionally (`--xlsx`) the Excel workbook too, `--csv` writes comma-separated instead of tab, `--quiet` suppresses output, and `--xlabel` / `--ylabel` set the plot axes; `pylcf-cli --help` lists every option.

A short version of this guide is available inside the program via "Show guide".

---

11. References

Methods used in the program and the software stack:

- **NNLS** (non-negative least squares): C. L. Lawson, R. J. Hanson, *Solving Least Squares Problems*, SIAM (1995).
- **SLSQP** (for the *convex* fit): D. Kraft, *A Software Package for Sequential Quadratic Programming*, DLR-FB 88-28 (1988).
- **Bootstrap**: B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall (1993).
- **Related LCF software (XAS):** B. Ravel, M. Newville, *ATHENA, ARTEMIS, HEPHAESTUS: data analysis for X-ray absorption spectroscopy using IFEFFIT*, J. Synchrotron Rad. **12**, 537–541 (2005); M. Newville, *Larch: An Analysis Package for XAFS and Related Spectroscopies*, J. Phys. Conf. Ser. **430**, 012007 (2013).
- **Software stack:** C. R. Harris et al., *Array programming with NumPy*, Nature **585**, 357–362 (2020); P. Virtanen et al., *SciPy 1.0*, Nat. Methods **17**, 261–272 (2020); J. D. Hunter, *Matplotlib: A 2D Graphics Environment*, Comput. Sci. Eng. **9**(3), 90–95 (2007).